A multi-agent system writing a manuscript

Contents

In this notebook we will use a multi-agent system to write a review about some arxiv papers. We will use smolagents to define agents with different responsibilities:

- A research assistant which can read arxiv papers and write manuscript sumaries.
- A reviewer which will provide constructive feedback given a text.
- A scientific writer which will incorporate the feedback and write a final manuscript.
- A printer which will print the final manuscript.
- A scheduler which distributes tasks to the other agents.

Note: For technical reasons, we only read the abstract. Of course, as in read life, it would be better to read the entire paper, but this exceeds token limits of SOTA open-weight LLMs.

```
from IPython.display import display, Markdown
from smolagents.agents import ToolCallingAgent, CodeAgent
from smolagents.prompts import CODE_SYSTEM_PROMPT
from smolagents import tool, LiteLLMModel
import os

# these functions are defined in arxiv_utilities.py
from arxiv_utilities import prompt_scadsai_llm, get_arxiv_metadata
```

```
C:\Users\rober\miniforge3\envs\genai-gpu\Lib\site-packages\pydantic\_internal\_config.py:345: UserWarning:
Valid config keys have changed in V2:
* 'fields' has been removed
  warnings.warn(message, UserWarning)
```

We configure to use a model provided on our institutional LLM server.

```
model =    "openai/meta-llama/Llama-3.3-70B-Instruct"
api_base = "https://llm.scads.ai/v1"
api_key =  os.environ.get('SCADSAI_API_KEY')
```

```
prompt = prompt_scadsai_llm
```

```
# comment this to show detailed output
from smolagents.utils import console
console.quiet = True
```

```
verbose = True
```

First, we define a function that generates a new agent with given name, description, system message, etc.

```
```

```
def create_agent(name, description, tools, model, api_base=None, api_key=None, system_message=None):
    """Create an agent that uses a given list of tools according to its system message."""
    model = LiteLLMModel(model_id=model,
                         api_base=api_base,
                         api_key=api_key)

    if system_message is None:
        system_message = CODE_SYSTEM_PROMPT
    else:
        system_message = CODE_SYSTEM_PROMPT + "\n" + system_message

    agent = CodeAgent(tools=tools, model=model, system_prompt=system_message)
    agent.name = name
    agent.description = description

    return agent
```

In this example we use a factory pattern for agents, to ensure that for every task, a new agent is created. This avoids very long prompts with former, irrelevant conversations.

```
def agent_factory(*args, **kwargs):
    def create_instance():
        return create_agent(*args, **kwargs)
    return create_instance
```

Research-assistant agent

We now define tools that can be called by the agent and an agent factory for the research-assistant. This agent can read arxiv meta-data, such as paper titles, authors and summaries.

```
@tool
def read_arxiv_paper(arxiv_url:str)->str:
    """Read the abstract of an arxiv-paper and return most important contents in markdown format.

    Args:
        arxiv_url: url of the Arxiv paper
    """
    if verbose:
        print(f"read_arxiv_paper({arxiv_url})")
    arxiv_id = arxiv_url.split("/")[-1]
    metadata = get_arxiv_metadata(arxiv_id)
    title = metadata["title"]
    summary = metadata["summary"]
    authors = ", ".join(metadata["authors"])

    return f"""## {title}
By {authors}

{summary}
"""

research_agent_factory = agent_factory(
    name="research-assistant",
    description="Scientific assistant who can read a paper and provide a summary of it.",
    system_message="""You will be tasked to read paper(s) and provide a summary.
    Write a very detailed manuscript about 1000 words outlining the major messages and limitations of a given
paper.""",
    tools=[read_arxiv_paper],
    model=model,
    api_base=api_base,
    api_key=api_key,
)
```

Scientific writer

We also define a scientific writer agent which can rewrite a manuscript by incorporating given feedback.

```
@tool
def improve_manuscript(manuscript:str, feedback:str)->str:
    """Can improve a given manuscript text according to defined feedback.

    Args:
        manuscript: The complete manuscript text to improve
        feedback: feedback to incorporate
    """
    if verbose:
        short = manuscript[:100]
        num_chars = len(manuscript)
        num_lines = len(manuscript.split("\n"))
        short_feedback = feedback[:100]
        num_chars_feedback = len(short_feedback)
        print(f"improve_manuscript({short}...[{num_chars} chars, {num_lines} lines], {short_feedback})...
[{num_chars_feedback} chars]")
    return prompt(f"""Improve a manuscript by incorporating given feedback.

## Manuscript

{manuscript}

## Feedback

{feedback}

## Your task

Improve the manuscript above by incorporating the feedback.
Do not shorten it!
Do not remove important details!
Use markdown links to cite sources.
Do not make up references!
Return the updated manuscript in markdown format only.
""")

scientific_writer_factory = agent_factory(
    name="scientific-writer",
    description="Scientific writer who improves manuscripts.",
    system_message="""You will be tasked to rewrite a text by incorporating given feedback.""",
    tools=[improve_manuscript],
    model=model,
    api_base=api_base,
    api_key=api_key,
)
```

```
## for testing:
#research_agent_factory().run("Read arxiv paper 2211.11501 and tell me the most important content in one
sentence.")
```

Reviewer agent

Next we define an LLM-based tool that can generate feedback for a given manuscript. Note: We are using the same LLM/server here as the agents use under the hood. This not necessary. One might use different LLMs for different tasks.

```
@tool
def review_text(manuscript:str)->str:
    """Reviews text and provides constructive feedback

    Args:
        manuscript: complete original manuscript text to review.
    """
    if verbose:
        short = manuscript[:100]
        num_chars = len(manuscript)
        num_lines = len(manuscript.split("\n"))
        print(f"review_text({short}...[{num_chars} chars, {num_lines} lines])")
    feedback = prompt(f"""
You are a great reviewer and you like to provide constructive feedback.
If you are provided with a manuscript, you formulate feedback specifically for this manuscript.
Your goal is to guide the author towards writing a great manuscript.
Hence, provide feedback like these examples but focus on what makes sense for the given manuscript:
* a scientific text with a short and descriptive title,
* a scientific text with markdown sub-sections (# title, ## headlines, ...) avoiding bullet points,
* structured in sub-sections by content, e.g. introduction, recent developments, methods, results, discussion,
future work, ...
* text using high-quality scientific language,
* proper citations mentioning the first author et al. using markdown links to original paper urls (do not make
up references!),
* avoid mentioning "the paper" and use proper markdown-link-citations instead,
* a clear abstract at the beginning of the text, and conclusions at the end

## Manuscript
This is the manuscript you are asked to review:

{manuscript}

## Your task
Provide constructive feedback to the manuscript above.
""")
    return feedback

reviewer_agent_factory  = agent_factory(
    name="reviewer",
    description="A reviewer who gives constructive feedback",
    tools=[review_text],
    model=model,
    api_base=api_base,
    api_key=api_key,
)
```

Printer agent

As it was hard to make the system return the final manuscript, we define an agent who is supposed to print the final manuscript. We can use this strategy to secretly also write the manuscript to a file.

```
@tool
def print_manuscript(manuscript:str)->str:
    """Prints a manuscript out provided in markdown format.

    Args:
        manuscript: An original manuscript text to print in markdown format containing all line breaks,
headlines, etc
    """
    from IPython.display import display, Markdown
    display(Markdown(manuscript))

    with open("manuscript.md", "w") as file:
        file.write(manuscript)

    return "The manuscript was printed."

printer_agent_factory  = agent_factory(
    name="printer",
    description="A professional printing expert who will print markdown-formatted text.",
    tools=[print_manuscript],
    model=model,
    api_base=api_base,
    api_key=api_key,
)
```

Scheduler agent

The scheduler is given a team of agents and can choose between them. For every task, it creates a new agent using the respective factory.

```
team = [research_agent_factory, reviewer_agent_factory, scientific_writer_factory, printer_agent_factory]

@tool
def distribute_sub_task(task_description:str, assistant:str)->str:
    """Prompt an assistant to solve a certain task.

    Args:
        task_description: Detailed task description, to make sure to provide all necessary details. When
handling text, hand over the complete original text, unmodified, containing all line-breaks, headlines, etc.
        assistant: name of the assistant that should take care of the task.
    """
    for t_factory in team:
        t = t_factory()
        if t.name == assistant:
            if verbose:
                print("".join(["-"]*80))
                short = task_description[:100]
                num_chars = len(task_description)
                print(f"| I am asking {assistant} to take care of: {short}...[{num_chars} chars]")

            # execute the task
            result = t.run(task_description)

            if verbose:
                short = result[:100]
                num_chars = len(result)
                print(f"| Response was: {short}...[{num_chars} chars]")
                print("".join(["-"]*80))

            return result

    return "Assistant unknown"

team_description = "\n".join([f"* {t().name}: {t().description}" for t in team])

scheduler = create_agent(
    name="scheduler",
    tools=[distribute_sub_task],
    description="Scheduler splits tasks into sub-tasks and distributes them.",
    system_message=f"""
You are an editor who has a team of assistants. Your task is to write a manuscript together with your team.

# Team
Your assistants can either read and summarize papers for you, or review text you wrote and provide feedback.

Your team members are:
{team_description}

# Typical workflow
A typical workflow is like this:
* Read papers
* Summarize them in a first manuscript draft
* Review the manuscript
* Incorporate review feedback to improve the manuscript
* Print the final manuscript in markdown format.

# Hints
When distributing tasks, make sure to provide all necessary details to the assistants.
Never shorten text when giving tasks to assistants. Provided them with the full manuscript text.

# Your task
Distribute tasks to your team. Goal is to have a great scientific manuscript.
""",
    model=model,
    api_base=api_base,
    api_key=api_key,
)
```

Writing a manuscript

Finally, we can ask the scheduler to distribute sub-tasks to the agents and produce the final result. Note that the task description is generic. It does not mention what the manuscript should be about. The system has to figure this out by reading the online resources.

```
manuscript = scheduler.run("""
Please take care of ALL the following tasks:
* Read these papers and summarize them
  * https://arxiv.org/abs/2211.11501
  * https://arxiv.org/abs/2308.16458
  * https://arxiv.org/abs/2411.07781
  * https://arxiv.org/abs/2408.13204
  * https://arxiv.org/abs/2406.15877
* Combine the information gained above and write a manuscript text about the papers,
* Afterwards, review the manuscript to get constructive feedback
* Use the feedback to improve the manuscript
* Print the final manuscript
""")
```

```
--------------------------------------------------------------------------------
| I am asking research-assistant to take care of: Summarize the paper https://arxiv.org/abs/2211.11501...[52
chars]
read_arxiv_paper(https://arxiv.org/abs/2211.11501)
| Response was: The paper introduces the DS-1000 benchmark, a reliable and challenging evaluation platform for
data ...[778 chars]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| I am asking research-assistant to take care of: Summarize the paper https://arxiv.org/abs/2308.16458...[52
chars]
read_arxiv_paper(https://arxiv.org/abs/2308.16458)
| Response was: The paper introduces BioCoder, a benchmark for evaluating the performance of large language
models i...[644 chars]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| I am asking research-assistant to take care of: Summarize the paper https://arxiv.org/abs/2411.07781...[52
chars]
read_arxiv_paper(https://arxiv.org/abs/2411.07781)
| Response was: The paper proposes RedCode, a benchmark for evaluating the safety of code agents, and presents
empir...[289 chars]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| I am asking research-assistant to take care of: Summarize the paper https://arxiv.org/abs/2408.13204...[52
chars]
read_arxiv_paper(https://arxiv.org/abs/2408.13204)
| Response was: The paper introduces the DOMAINEVAL benchmark for evaluating LLMs' code generation capabilities
acro...[388 chars]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| I am asking research-assistant to take care of: Summarize the paper https://arxiv.org/abs/2406.15877...[52
chars]
read_arxiv_paper(https://arxiv.org/abs/2406.15877)
| Response was: The paper introduces BigCodeBench, a new benchmark for evaluating LLMs' ability to solve
challenging...[538 chars]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| I am asking reviewer to take care of: Review the manuscript: The recent papers introduce several new
benchmarks for evaluating the perform...[777 chars]
review_text(The recent papers introduce several new benchmarks for evaluating the performance of large language
...[754 chars, 1 lines])
| Response was: The manuscript provides a good overview of recent benchmarks for evaluating large language
models in...[231 chars]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| I am asking scientific-writer to take care of: Improve the manuscript: The recent papers introduce several
new benchmarks for evaluating the perfor...[1033 chars]
improve_manuscript(The recent papers introduce several new benchmarks for evaluating the performance of large
language ...[754 chars, 1 lines], The manuscript provides a good overview of recent benchmarks for evaluating
large language models in)... [100 chars]
| Response was: # Evaluating Large Language Models in Code Generation: Recent Benchmarks and Future Directions
## Ab...[2595 chars]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
| I am asking printer to take care of: Print the manuscript: # Evaluating Large Language Models in Code
Generation: Recent Benchmarks and F...[2617 chars]
```

Evaluating Large Language Models in Code Generation: Recent Benchmarks and Future Directions

Abstract

The recent introduction of several new benchmarks has significantly advanced the evaluation of large language models (LLMs) in generating code. This manuscript provides an overview of these benchmarks, including BigCodeBench, DS-1000, BioCoder, RedCode, and DOMAINEVAL, and discusses their implications for the development of LLMs.

Introduction

The recent papers introduce several new benchmarks for evaluating the performance of large language models (LLMs) in generating code. These benchmarks include BigCodeBench, DS-1000, BioCoder, RedCode, and DOMAINEVAL. The results of these papers show that LLMs are not yet capable of following complex instructions to use function calls precisely and struggle with certain tasks such as cryptography and system coding. However, they also demonstrate the potential of LLMs in generating bioinformatics-specific code and highlight the importance of domain-specific knowledge.

Discussion

Overall, these benchmarks provide a challenging and reliable evaluation platform for data science code generation models and emphasize the need for further research and development. The results of these benchmarks have significant implications for the development of LLMs, highlighting the need for improved performance in following complex instructions and generating code for specific domains. Furthermore, the benchmarks demonstrate the potential of LLMs in generating high-quality code for certain tasks, such as bioinformatics, and emphasize the importance of incorporating domain-specific knowledge into LLMs.

Conclusion

In conclusion, the recent benchmarks for evaluating LLMs in code generation have provided significant insights into the capabilities and limitations of these models. The results of these benchmarks highlight the need for further research and development to improve the performance of LLMs in generating code, particularly in areas such as cryptography and system coding. However, they also demonstrate the potential of LLMs in generating high-quality code for specific domains, such as bioinformatics, and emphasize the importance of incorporating domain-specific knowledge into these models.

```
| Response was: The manuscript has been printed....[32 chars]
--------------------------------------------------------------------------------
```

Exercise

Modify the task above and challenge the system. Provide URLs to arxiv papers of different topic, e.g. to papers you know well. Also provide urls to paper which don’t exist.

Question the output: How much of the text is made up, how much was actually content of the paper (abstracts)?

By Robert Haase
Last updated on 2025-11-10.